

C O D E X A A I

DISTRIBUTED AI CODE GENERATION PLATFORM

Master Design Book (Textbook Edition)

An In-Depth Line-by-Line Code and Architectural Analysis Guide

Author: Md Irfan

Software Engineer | Java Microservices | Cloud-Native AI Architectures

Inspired by Lovable.dev Platform Architecture

Date: June 2026 | Manual Version: 3.0.0-RELEASE

1. Preface & Manual Structure

Table of Contents & Core Document Navigation

This master textbook reference provides a comprehensive, line-by-line guide and technical analysis of the Codexa AI platform. Codexa AI is a distributed, event-driven system built using Spring Boot and Spring Cloud, capable of parsing natural language directives, compiling complete web applications, and executing them on-demand in isolated sandbox pods. By integrating code generators, Kafka saga brokers, MinIO file-stores, and Eureka dynamic routes, Codexa AI reproduces the responsive code creation experience of Lovable.dev.

To help you master both the business logic of this repository and the underlying web technologies, this handbook contains detailed explanations of core concepts: from reactive gateways, service discovery registries, and Config Server Git pipelines, to Feign security contexts, transactional Kafka sagas, relational index optimizations, MinIO storage sidecars, and Kubernetes dynamic pod allocations. This ensures the document is fully populated and leaves no blank spaces, providing exhaustive reference material.

This handbook is organized into six major technical topics, reordered as requested to present data layers first:

- **Topic 1:** System Scope, Problem Statement & Functional/Non-Functional Requirements
- **Topic 2:** API Design: Gateway Routing Configs & Service Controllers Endpoint Matrix
- **Topic 3:** Entity Design: User JPA Model & Database Schema
- **Topic 4:** Deploy: GKE Cluster Setup & Dynamic Multi-Cloud Operations Manual
- **Topic 5:** HLD Conceptual Architecture: Decoupled Mappings & Workflow Flows
- **Topic 6:** Low-Level Design (LLD) Blueprint & Step-by-Step Code Walkthroughs

Topic 1: System Scope & Problem Statement

Developer Bottlenecks & Platform Solutions

Traditional software engineering involves a high-overhead workflow: checking out repositories, setting up dependencies, configuring databases, checking port collisions, and coordinating container build outputs. When non-technical users or rapid prototypers want to build software, these hurdles make the process impossible. Even for senior developers, bootstrapping an environment to test a minor change or prototype a new service wastes significant time. Additional complications arise from local system dependencies, matching database credential setups, operating system-specific runtimes, and network access policies that vary between local machines and cloud environments.

Codexa AI solves this problem by automating the entire software development life cycle. By inputting a single prompt, the platform determines dependencies, initializes directories, generates index templates, sets up API bindings, and spins up an isolated sandbox. The system translates text commands into real codebase changes, compiles the files, deploys the resulting app in a container, and routes traffic in real-time, removing the manual git pull/compile cycle. This manual details how each component in this pipeline fits together to provide a robust, cloud-native SaaS experience.

Underneath, this platform leverages Spring Boot as a core framework, utilizing Spring Cloud Gateway for routing, Eureka for service lookup, PostgreSQL for relational metadata, MinIO for S3-compliant object persistence, Kafka for asynchronous saga event routing, and Kubernetes for sandboxed runner deployments. This architecture ensures that user workspaces are isolated, high-performance, and vertically scalable without configuration overhead.

Topic 1: Functional Requirements Spec

Client-Facing Features & Core Platform Capabilities

The platform implements specific functional scopes to maintain user workflows. These are structured as follows:

1. Project Workspace_INITIALIZER: Allows users to create workspaces based on pre-defined templates (React, static HTML, CSS templates) with soft delete capabilities. It manages the metadata registry, database listings, and initial bucket partitions. This ensures that every project starts with a clean, functional workspace.
2. AI Chat Streamer Endpoint: Provides a Server-Sent Events (SSE) streaming interface that queries OpenAI, resolves workspace context, parses structured tags, and maps thoughts and file mutations back to the client interface in real-time. This provides an interactive, responsive chat experience.
3. Workspace File Editor: Exposes manual code editing, folder deletions, directory creations, and binary uploads via standard REST APIs proxying directly to MinIO storage. It enables developers to edit the generated code manually at any point, providing full control over the codebase.
4. Pre-Warmed Preview Claimer: Orchestrates sandbox deployments. Grabs pre-warmed idle Kubernetes pods, links them to project IDs, updates routing configurations, and boots development servers to display live outputs within seconds, reducing wait times.
5. User Stripe Billing Integration: Exposes price plans, registers user Stripe Customer IDs, processes checkout sessions, and monitors payment events via secured webhooks to restrict premium features based on subscription states and handle payment events.

Functional Focus: The core user experience is centered around the conversation. Every prompt mutation is converted to a structured AST (Abstract Syntax Tree) edit, written to MinIO, and reflected instantly inside the sandboxed preview emulator, removing the manual git pull/compile cycle.

Topic 1: Non-Functional Requirements Spec

Performance, Scalability & Security Constraints

To remain viable in production, Codexa AI enforces strict operational constraints across the system topology:

1. Sub-Second SSE Handshakes: Server-Sent Events streaming must establish connection handshakes and output first-thought tokens in under 500ms. This requires reactive Netty gateway routing and immediate thread handover to prevent

connection timeouts.

2. Sandbox Isolation Boundaries: Preview runner pods operate under strict Kubernetes network policies. Cross-pod traffic and access to the main cluster control plane are blocked to prevent untrusted code execution from breaching the system.
3. Kafka Transactional Sagas: File modifications occur via Kafka topics. Consumers check transactional status keys in a deduplication table to prevent duplicate updates or race conditions from out-of-order event delivery, maintaining data consistency.
4. Automatic Sandbox Recycling: Preview environments run on idle timeouts. If no requests target a preview for 6 hours, a cron job terminates the pod and returns it to the pre-warmed pool, preventing resource exhaustion and reducing cluster costs.
5. Local Service Authentication: Each microservice decrypts JWT bearer tokens locally via standard shared libraries, preventing Gateway bottle-necks and ensuring stateless vertical scaling of backend runtimes across multiple availability zones.

Operational Excellence: By keeping the Gateway decoupled from database calls and validating JWT signatures locally on each microservice, the system can scale horizontally to handle thousands of concurrent requests.

Topic 2: API Design: Gateway Routing Configs

Gateway Routing Specifications & Path Mapping

The API Gateway routes traffic based on URL path patterns to internal microservice instances. The following table specifies the route configurations, target service mappings, and path filters:

Routing logic is defined declaratively. When the Gateway receives a request, it matches the path predicate, applies configured filters (such as path stripping or header mutations), and forwards the request to the service instance resolved via Eureka. This decouples public endpoints from internal deployment details.

In addition, the Gateway manages path stripping dynamically via filter annotations, ensuring the internal microservices receive clean, context-free request URIs.

- **/api/auth/****: Routes public user actions, registration, and log-ins to the Account Service (Port 9050).
- **/api/billing/****: Routes session checkouts and active user subscription queries to the Account Service (Port 9050).
- **/api/projects/****: Routes project CRUD details, member invites, and workspace operations to the Workspace Service (Port 9020).
- **/api/chat/****: Routes SSE prompt stream mapping endpoints to the Intelligence Service (Port 9030).

Topic 2: API Design: Account Auth APIs

Authentication REST Controller specifications

Endpoints under the `/auth` path control identity and profiles:

These endpoints handle registration, login, and internal service calls. Password hashing and JWT generation occur inside the Account Service. Internal endpoints allow other microservices (such as the Workspace Service) to retrieve user metadata securely over Feign without exposing these endpoints to the public internet.

Security context validations are skipped for the `/signup` and `/login` paths, enabling public signup and access control bootstrapping.

- **POST /auth/signup**: Body: SignupRequest {email, password, name}. Action: registers user credentials, maps default Free subscription records, returns AuthResponse DTO.
- **POST /auth/login**: Body: LoginRequest {email, password}. Action: verifies credentials, generates JWT signature, returns AuthResponse DTO.
- **GET /internal/v1/user/{id}**: Header: Auth Bearer. Action: returns User DTO metadata matching path ID variable. (Feign endpoint)
- **GET /internal/v1/users/by-email**: Header: Auth Bearer. Params: email. Action: returns User DTO matching query string. (Feign endpoint)

Topic 2: API Design: Account Billing APIs

Stripe checkouts & Webhooks payment processing

Payment and subscription endpoints integrate with Stripe:

These endpoints process checkout sessions, manage subscriptions, and handle Stripe webhooks. When a user requests a subscription change, the system generates a Stripe checkout URL. Stripe updates are processed asynchronously via a webhook endpoint that verifies the request signature before modifying user limits in the database.

If a subscription payment fails, Stripe events automatically trigger downgrade events, updating PostgreSQL schemas.

- **GET /subscription**: Header: Auth Bearer. Action: queries active subscription parameters, plan metadata, current limits.
- **POST /payments/checkout**: Header: Auth Bearer. Body: CheckoutRequest {priced}. Action: generates Stripe checkout session redirection link.
- **POST /payments/portal**: Header: Auth Bearer. Action: returns Stripe customer portal url for billing adjustments.
- **POST /webhooks/payment**: Public. Headers: Stripe-Signature. Action: validates signature, processes checkout session

metadata, updates subscription statuses.

Topic 2: API Design: Workspace Metadata APIs

Project CRUD controllers specs & permissions validation

Workspace Service endpoints manage project metadata, verifying permissions via project member tables:

These endpoints handle workspace creation, retrieval, updates, and deletion. Method-level security checks ensure that users can only access projects they own or are members of, returning a 403 Forbidden response if verification fails.

Soft deletion limits the database updates to editing the project's `deleted\_at` field, enabling simple recoveries.

- **GET /projects:** Header: Auth Bearer. Action: retrieves a JSON summary list of projects owned by or shared with the user.
- **GET /projects/{id}:** Header: Auth Bearer. Action: retrieves details of a specific project, validating member RBAC role constraints.
- **POST /projects:** Header: Auth Bearer. Body: ProjectRequest {name, isPublic}. Action: validates plan constraints, inserts project records, returns project payload.
- **PATCH /projects/{id}:** Header: Auth Bearer. Body: ProjectRequest. Action: updates project parameters, such as name or visibility status.

Topic 2: API Design: Workspace File APIs

File tree queries & MinIO folder mirrors

Endpoints under the `/projects/{projectId}/files` path handle project files:

These endpoints generate the workspace file tree structure and download file contents. When a user opens a project, the client calls `/files` to retrieve a nested JSON tree of the workspace, fetching raw file text contents directly from MinIO as needed.

The dynamic zip compilation pulls files from MinIO, packages them into an in-memory zip archive, and streams the output.

- **GET /projects/{projectId}/files:** Header: Auth Bearer. Action: returns file tree metadata matching project files registry.
- **GET /projects/{projectId}/files/content:** Header: Auth Bearer. Params: path. Action: retrieves file contents from MinIO storage.
- **GET /projects/{projectId}/download-zip:** Header: Auth Bearer. Action: packages directory files into a zip download stream.

Topic 2: API Design: AI Chat Sessions APIs

Intelligence service prompt streaming endpoints

The Intelligence Service endpoints handle code generation requests and chat history:

The `/stream` endpoint establishes an SSE connection. As the LLM generates updates, tokens are streamed back to the client. After the stream completes, the service parses the code edits and publishes file update events to Kafka.

This ensures the dialogue interface remains dynamic and interactive without blockages.

- **POST /chat/stream:** Header: Auth Bearer. Body: ChatRequest {message, projectId}. Action: executes prompting loop, returns SSE stream chunks.
- **GET /chat/projects/{projectId}:** Header: Auth Bearer. Action: returns a list of ChatResponse messages mapping historical logs.

Topic 2: API Design: Preview Sandbox Control APIs

Claiming sandbox pod runtimes & logging emulations

Dynamic sandbox endpoints manage isolated preview environments:

These endpoints claim, monitor, and stream logs from the sandboxed previews. The deploy endpoint initiates the pod claiming process and returns the subdomain URL used to access the running application.

Logs are read by connecting an active InputStream to the runner container stdout.

- **POST /projects/{id}/deploy**: Header: Auth Bearer. Action: claims an idle runner pod, launches sync loop, returns the routing URL.
- **GET /previews/{id}/status**: Header: Auth Bearer. Action: returns current status of compiling sandboxes.
- **GET /previews/{id}/logs**: Header: Auth Bearer. Action: streams stdout dev logs from the running sandbox back to the client.

Topic 3: Entity Design: User JPA Model

Account Service JPA entity attributes & annotations analysis

The User entity represents the base credential table inside the database. It stores user credentials, Stripe customer IDs, and soft-delete auditing fields. Key annotations explained step-by-step:

- `@Entity` designates this class as a JPA database entity mapped to a relational table.
- `@Table(name = 'users')` specifies the target database table name.
- `@Id` registers the primary key field, and `@GeneratedValue` delegates auto-incrementing ID creation to the database.
- `@Column(unique = true)` enforces uniqueness on the stripe customer ID field.
- `@CreationTimestamp` and `@UpdateTimestamp` automate record audit tracking on creation and updates.

[User.java (JPA Model)]

```
@Getter @Setter @Entity
@Table(name = "users")
@AllArgsConstructor @Builder
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    String username;
    String password;
    String name;

    @Column(unique = true)
    String stripeCustomerId;

    @CreationTimestamp
    Instant createdAt;

    @UpdateTimestamp
    Instant updatedAt;

    Instant deletedAt; // Soft delete timestamp
}
```

Topic 3: Entity Design: Plan JPA Model

Subscription tiers constraints & properties annotations analysis

The Plan entity defines the resource limits and pricing details for each subscription tier. Key annotations and attributes explained step-by-step:

- `@Table(name = 'plans')` defines the target relational table storing tier metadata.
- `@Column(unique = true)` enforces that price IDs mapped from Stripe are unique across the table.
- `maxProjects` and `maxTokensPerDay` columns store usage quotas checked on workspace creation and chat executions.
- `active` is a boolean flag indicating whether the pricing plan is currently available for purchase.

[Plan.java (JPA Model)]

```
@Entity @Getter @Setter
@Table(name = "plans")
public class Plan {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;
    String name;

    @Column(unique = true)
    String stripePriceId;

    Integer maxProjects;
    Integer maxTokensPerDay;
    String maxPreviews;
    Boolean unlimitedAi;
    Boolean active;
}
```


Topic 3: Entity Design: Subscription JPA Model

User subscription states & relationships annotations analysis

The Subscription entity maps users to plans and tracks billing boundaries. Key annotations and mappings explained step-by-step:

- `@ManyToOne(fetch = FetchType.LAZY)` defines a foreign key relationship to the User table. Lazy fetching ensures user details are only loaded from the database when accessed, optimizing query performance.
- `@JoinColumn(name='user_id', nullable=false)` maps the foreign key column name and enforces that every subscription must refer to a valid user.
- `@Enumerated(EnumType.STRING)` specifies that the `SubscriptionStatus` enum values (ACTIVE, PAST_DUE) are stored as text strings in the database, ensuring readability.

[Subscription.java (JPA Model)]

```
@Entity @Getter @Setter
@Table(name = "subscriptions")
public class Subscription {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name="user_id", nullable=false)
    User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name="plan_id", nullable=false)
    Plan plan;

    String stripeSubscriptionId;
    Instant currentPeriodStart;
    Instant currentPeriodEnd;
    Boolean cancelAtPeriodEnd = false;

    @Enumerated(value=EnumType.STRING)
    SubscriptionStatus status;
}
```

Topic 3: Entity Design: Project JPA Model

Workspace metadata database mappings & index schemas annotations analysis

The Project entity stores project metadata. Database indexes are optimized to speed up workspace queries and handle soft delete checks efficiently. Key annotations and indexes explained:

- `@Table` includes an array of `@Index` annotations. For example, `idx_projects_updated_at_desc` configures a compound index on the `updated_at DESC` and `deleted_at` fields, optimizing workspace listings.
- `deleted_at` timestamp stores the soft delete timestamp. In queries, the service filters out records where `deleted_at IS NOT NULL` instead of deleting rows, allowing for workspace recovery.

[Project.java (JPA Model)]

```

@Entity @Getter @Setter
@Table(name = "projects", indexes = {
    @Index(name = "idx_projects_updated_at_desc", columnList = "updated_at DESC, deleted_at"),
    @Index(name = "idx_projects_deleted_at_updated_at_desc", columnList = "deleted_at, updated_at DESC"),
    @Index(name = "idx_projects_deleted_at", columnList = "deleted_at")
})
public class Project {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    @Column(nullable = false)
    String name;

    Boolean isPublic = false;

    @CreationTimestamp
    Instant createdAt;

    @UpdateTimestamp
    Instant updatedAt;

    Instant deletedAt;
}

```

Topic 3: Entity Design: ProjectFile JPA Model

Workspace directories metadata & storage paths mapping annotations analysis

The ProjectFile entity maps workspace file paths to their MinIO storage keys. Key annotations and attributes explained step-by-step:

- `@ManyToOne` maps the parent project relationship, creating a foreign key mapping to the `projects` table.
- `path` stores the relative filepath inside the workspace (e.g. `src/components/Button.jsx`).
- `minioObjectKey` stores the unique key pointing to the file content in MinIO. This separates metadata queries from large binary operations, allowing the system to scale.

[ProjectFile.java (JPA Model)]

```

@Entity @Table(name = "project_files")
@Getter @Setter @Builder
@AllArgsConstructor @NoArgsConstructor
public class ProjectFile {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "project_id", nullable = false)
    Project project;

    @Column(nullable = false)
    String path;

    String minioObjectKey;

    @CreationTimestamp
    Instant createdAt;

    @UpdateTimestamp
    Instant updatedAt;
}

```

Topic 3: Entity Design: ProjectMember JPA Model & Keys

Workspace composite identity memberships RBAC rules annotations analysis

ProjectMember tracks project sharing and access controls, using a composite primary key to link user and project records. Key annotations explained step-by-step:

- `@EmbeddedId` registers the composite primary key class `ProjectMemberId` inside the entity.
- `@MapsId("projectId")` maps the `project` relationship to the corresponding field in the composite key.
- The composite primary key class implements `Serializable` and overrides `equals` and `hashCode` to ensure hibernate can

resolve keys in cache sessions.

[ProjectMember.java & ProjectMemberId.java]

```
@Entity @Table(name = "project_members") @Getter @Setter
public class ProjectMember {
    @EmbeddedId ProjectMemberId id;
    @ManyToOne @MapsId("projectId") Project project;
    @Enumerated(EnumType.STRING) @Column(nullable = false) ProjectRole projectRole;
    Instant invitedAt;
    Instant acceptedAt;
}

@Embeddable @Getter @Setter @EqualsAndHashCode
public class ProjectMemberId implements Serializable {
    @Column(name = "project_id") Long projectId;
    @Column(name = "user_id") Long userId;
}
```

Topic 3: Entity Design: ChatSession JPA Model & Keys

Intelligence dialogue context session composite keys annotations analysis

The ChatSession entity manages active chat sessions. It uses a composite key linking user and project to isolate conversation contexts. Key annotations explained step-by-step:

- `@EmbeddedId` registers the `ChatSessionId` class mapping containing the compound primary key.
- `@MapsId` binds the parent database mappings to project and user tables, ensuring relational integrity.
- Soft delete timestamps (`deleted_at`) allow users to clear their chat history without losing context audits.

[ChatSession.java & ChatSessionId.java]

```
@Entity @Table(name = "chat_sessions") @Getter @Setter
public class ChatSession {
    @EmbeddedId ChatSessionId id;
    @CreationTimestamp @Column(nullable = false, updatable = false) Instant createdAt;
    @UpdateTimestamp Instant updatedAt;
    Instant deletedAt;
}

@Embeddable @Getter @Setter @EqualsAndHashCode
public class ChatSessionId implements Serializable {
    @Column(name = "project_id") Long projectId;
    @Column(name = "user_id") Long userId;
}
```

Topic 3: Entity Design: ChatMessage JPA Model

Historical dialogue messages & consumed tokens mapping annotations analysis

The ChatMessage entity stores chat history and tracks token usage. It references the parent ChatSession using a composite foreign key mapping. Key annotations and details explained:

- `@JoinColumns` configures a compound foreign key. It maps both columns of the composite parent key (`project_id` and `user_id`) to maintain referential integrity with the `chat_sessions` table.
- `role` stores the message type (USER or ASSISTANT) as an enum string to define context paths in chat history.
- `content` is mapped as `@Column(columnDefinition = 'text')` to support large text outputs from the LLM.

[ChatMessage.java (JPA Model)]

```

@Entity @Table(name = "chat_messages") @Getter @Setter
public class ChatMessage {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumns({
        @JoinColumn(name="project_id", referencedColumnName="project_id", nullable=false),
        @JoinColumn(name="user_id", referencedColumnName="user_id", nullable=false)
    })
    ChatSession chatSession;

    @Column(columnDefinition = "text", nullable = false) String content;
    String toolCalls; // JSON string of parsed tool calls
    Integer tokensUsed = 0;
    @CreationTimestamp Instant createdAt;

    @Enumerated(EnumType.STRING) @Column(nullable=false) MessageRole role;
}

```

Topic 3: Entity Design: ChatEvent JPA Model

LLM parsing actions logging & transactional status tracking annotations analysis

The ChatEvent entity logs individual file actions generated during a chat. It records the `sagald` to map file update events to the Kafka pipeline. Key annotations explained step-by-step:

- `@ManyToOne` maps the relationship to the corresponding `ChatMessage` that generated the action.
- `sequenceOrder` tracks the execution order of file changes inside a single AI response.
- `sagald` maps the event to the asynchronous Kafka pipeline, allowing the system to track the status of file writes (PENDING, SUCCESS, FAILED).

[ChatEvent.java (JPA Model)]

```

@Entity @Table(name = "chat_events") @Getter @Setter
public class ChatEvent {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(nullable = false) ChatMessage chatMessage;
    @Column(nullable = false) Integer sequenceOrder;
    @Enumerated(EnumType.STRING) @Column(nullable = false) ChatEventType type;
    @Column(columnDefinition = "text") String content;
    String filePath;
    String metadata;
    String sagaId;
    @Enumerated(EnumType.STRING) ChatEventStatus status;
}

```

Topic 3: Entity Design: ProcessedEvent Saga Model

Kafka Saga event consumption idempotence registries annotations analysis

The ProcessedEvent entity acts as a deduplication registry to enforce idempotency in Kafka event consumption. Processed event IDs are written here to prevent reprocessing in case of event redelivery. Key annotations:

- `@Id` sets the `event\_id` field as the primary key. When a new event arrives, the consumer queries this table using the `sagald` to verify if it has already been processed.
- `processed\_at` records the timestamp of the event write, enabling automatic cleanup of old transactions.

[ProcessedEvent.java (JPA Model)]

```
@Entity
@Table(name = "processed_events")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor
public class ProcessedEvent {
    @Id
    @Column(name = "event_id")
    String eventId;

    @Column(name = "processed_at", nullable = false)
    LocalDateTime processedAt;
}
```

Topic 4: Deploy: GKE Cluster Setup

Google Cloud cluster setup & namespace credentials allocations

Deploying Codexa AI in production uses Google Kubernetes Engine (GKE) to manage scaling and sandbox isolation. Connect your local terminal to the GKE cluster using the Google Cloud SDK:

```
gcloud container clusters get-credentials lovable-me-cluster --region asia-south1 --project project-9ca46dba-039d-4038-bfe
```

Create isolated namespaces to separate core system services from dynamic user sandbox previews:

```
kubectl create namespace lovable-core
kubectl create namespace lovable-previews
```

This isolation allows you to apply strict network policies that block runner pods from accessing the core platform services or databases, securing the cluster against untrusted user code.

Topic 4: Deploy: GKE Secrets & Database persistent sets

Setting up environment configurations & launching stateful DBs

Apply environment secrets and configmaps, then deploy the stateful storage infrastructure:

```
kubectl create secret generic app-secrets --from-env-file=.env -n lovable-core
kubectl apply -f k8s/configmaps/ -n lovable-core
```

Deploy PostgreSQL, MinIO, Redis, and Apache Kafka. These are configured as StatefulSets with Persistent Volume Claims (PVCs) to ensure data persists across container restarts:

```
kubectl apply -f k8s/databases/ -n lovable-core
```

Verify database pod health before deploying backend services:

```
kubectl get pods -n lovable-core -l app.kubernetes.io/part-of=databases
```

Topic 4: Deploy: GKE Services & Ingress routings setup

Deploying stateless microservices & Ingress routing rules

Deploy the stateless microservices (API Gateway, Account Service, Workspace Service, Intelligence Service):

```
kubectl apply -f k8s/services/ -n lovable-core
```

Install the NGINX Ingress controller using Helm or the provider manifest to provision a public IP:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.8.2/deploy/static/provider/cloud/deploy.yaml
```

Wait for the external LoadBalancer IP allocation and apply the ingress routing map:

```
kubectl get svc ingress-nginx-controller -n ingress-nginx -w
kubectl apply -f k8s/ingress.yaml -n lovable-core
```

Point your DNS records (A and wildcard CNAME) to the external Ingress LoadBalancer IP.

Topic 4: Deploy: Azure AKS cluster setups

Establishing AKS subscriptions & managed CSI storage classes

For Azure deployments, use the Azure CLI to connect to the AKS cluster and configure resources:

```
az login
az account set --subscription "Your-Azure-Subscription-ID"
```

```
az aks get-credentials --resource-group codexa-resources --name codexa-aks-cluster
```

Apply the environment variables and run configuration scripts to map dynamic storage volumes:

```
source k8s/infra/domains.env  
./scripts/configure-domains.sh
```

This configures Azure Managed Disks using the Azure CSI driver to provide persistent storage volumes.

Topic 4: Deploy: Production Scaling, HPAs & Telemetry configurations

Configuring pod autoscalers limits & Observability tracking metrics

Set horizontal pod autoscaling rules and track system telemetry using Zipkin and Spring Boot Actuators:

```
kubectl autoscale deployment runner-pool --cpu-percent=70 --min=5 --max=50
```

Monitor actuator trace nodes and collect sandbox metrics to optimize preview pod allocation limits.

Verify GKE and AKS image tags to align CI/CD workflows.

Topic 5: HLD Conceptual Architecture

Decoupled Microservices & Stateless Service Boundaries

At a conceptual high level, Codexa AI partitions operations across isolated, single-responsibility services. User connections route through Ingress routing boundaries. Standard static client files are served by the Frontend Service, while all API-bound backend requests proxy through the central Spring Cloud API Gateway. This separation of concerns allows us to update business logic, the billing engine, or the AI prompting system without affecting other components.

To guarantee sub-second routing of previews, the Gateway intercepts subdomain hosts. A wildcard ingress pattern maps preview requests directly to a dedicated Proxy Service, which queries a Redis cache containing active pod IP routes. This ensures that user previews are sandboxed inside isolated namespaces, while the core platform remains secure and insulated from developer code runtimes.

The services interact in a stateless manner. User sessions are verified via signed, cryptographically secure JWT tokens containing client credentials and subscription scopes. When a microservice receives a token, it decrypts it locally, fetches the user's role mapping, and checks method authorization without querying a central session database. This decoupled design allows the platform to achieve high availability, low latency, and simplified fault tolerance.

Topic 5: Conceptual System Topology

HLD Block Flow Mappings

The HLD defines the logical relationships between: Ingress Controller, Frontend React UI, Ingress Proxy, API Gateway, discovery registry node maps (Eureka), configuration server (Config Service), persistence repositories (PostgreSQL, MinIO), the dynamic runner namespaces, and the asynchronous Saga topic broker (Apache Kafka). By keeping persistence decoupled, services can scale independently. Workspace updates publish to the Saga topic broker, and consumer services modify workspaces, storage objects, and preview runners asynchronously.

The Config Service acts as the central settings manager, fetching configurations from a secured GitHub repository on startup. This service registry pattern (Eureka) allows Feign clients to communicate using abstract service names (e.g., `http://workspace-service/projects`) instead of hardcoded IPs. The Eureka server tracks active microservice instances via periodic heartbeats. If a service instance crashes, Eureka updates its registry, and the Gateway diverts traffic to healthy nodes, maintaining high system availability.

In addition, the Config Service supports dynamic profile switching (such as local development vs production GKE/AKS), enabling the same container images to run in any environment with different configuration parameters. This decouples application code from operational infrastructure details, simplifying deployments.

Architectural Pattern: By combining Spring Cloud Config Server and Eureka Discovery, we decouple infrastructure deployment layouts from application code, making local, staging, and GKE/AKS rollouts identical.

Topic 5: HLD Domain and Subdomain Routing Map

Core User and Preview Networking Paths

The ingress layer handles traffic routing based on the host domain:

1. Frontend React UI: Matches `codingshuttle.in``. The NGINX Ingress controller routes these calls to the stateless frontend container running the React build. This serves the landing page, chat interface, and workspace panels.
2. Backend API Gateway: Matches `api.codingshuttle.in``. Requests are routed to the Spring Cloud Gateway on port 8080. The Gateway validates user JWTs, rewrites paths, and proxies requests to `/api/account``, `/api/workspace``, and `/api/intelligence``.
3. Dynamic Sandboxed Previews: Matches `*.previews.codingshuttle.in``. These wildcard calls target the Proxy Service. When traffic hits the wildcard previews subdomain, the Proxy Service extracts the unique project ID prefix, queries the Redis cache to find the active Kubernetes pod IP in the `lovable-previews`` namespace, and proxies the HTTP request directly. This dynamic routing mechanism allows users to interact with their generated React web apps instantly without waiting for DNS propagation.

If no route mapping is found in the Redis cache, the Proxy Service returns a 404 error page prompting the user to deploy. This ensures the system degrades gracefully and secures the preview namespace against stale or inactive subdomains.

Routing Performance: The Proxy Service uses an asynchronous Node/Express proxy engine, fetching container IPs from Redis in under 2ms, enabling sub-second load times for claimed preview environments.

Topic 5: Stateful HLD Partitioning Design

Relational Data, Object Storage & Event Topics Mappings

SaaS data splits across three distinct storage tiers:

1. PostgreSQL: A relational database storing users, plans, subscriptions, project metadata, and audit logs. Relational integrity is enforced via foreign keys, cascading deletes, and unique constraints. Indexes are optimized for soft delete checks.
2. MinIO: S3-compatible object storage storing project files in specific paths (`projects/{id}/files/`). Rather than storing source files in database columns, they are saved as discrete S3 objects, allowing high-speed file streams and sync sidecar mounting.
3. Redis: An in-memory key-value database caching transient routing entries (domain-to-pod mappings) and user profiles, offloading load from PostgreSQL.

Kafka orchestrates the asynchronous Saga event pipeline. When the Intelligence Service requests a file write, it publishes a `FileStoreRequestEvent` to the `file-store-request-event` topic. The Workspace Service consumes the event, writes the file to MinIO, updates PostgreSQL, and registers the `sagald` in an idempotency table to prevent duplicate file modifications from event redelivery.

This partitioning ensures that read-heavy workloads (such as preview traffic routing lookups) hit the fast in-memory Redis cache, while write-heavy operations (file mutations) run asynchronously through Kafka, keeping the user interface responsive.

Topic 5: End-to-End Generation Workflow Flow

Dialogue and Deployment Sequence

The end-to-end prompt processing loop executes as follows:

1. SSE Connection: The client browser opens a Server-Sent Events connection to `/chat/stream` through the API Gateway, establishing a non-blocking reactive stream with the Intelligence Service.
2. Context Gathering: The Intelligence Service queries the Workspace Service via Feign to retrieve the project's file structure and folder lists. The `FileTreeContextAdvisor` formats this list and injects it into the prompt context.
3. LLM Querying: The prompt is sent to the OpenAI API. The model streams back tokens in real-time, which are forwarded to the client.
4. Stream Finalization: On completion, the service parses the response for `` tags. For each edit, it generates a unique `sagald` and publishes a `FileStoreRequestEvent` to Kafka.
5. Workspace Persistence: The Workspace Service consumes the Kafka event, writes the new file to MinIO, registers the transaction, and signals the claimed Kubernetes runner pod. The sidecar syncer container downloads the files from MinIO, and the dev server updates.

This workflow ensures that the client receives immediate feedback via the SSE stream, while the heavier file operations, compilations, and container synchronization execute asynchronously in the background.

Asynchronous Core: By offloading file editing and storage updates to Kafka topics, the user dialogue remains responsive. Streaming tokens are delivered without waiting for database operations to complete.

Topic 6: Low-Level Design (LLD) Blueprint

Overall Microservices & Network Routing Map

The overall Low-Level Design (LLD) blueprint details the system's operational components. It defines the networking ports, context paths, service boundaries, database mappings, and communication paths. This layout details how the system is deployed inside the cluster, ensuring that developers can reproduce the environment locally or deploy it to a production Kubernetes cluster.

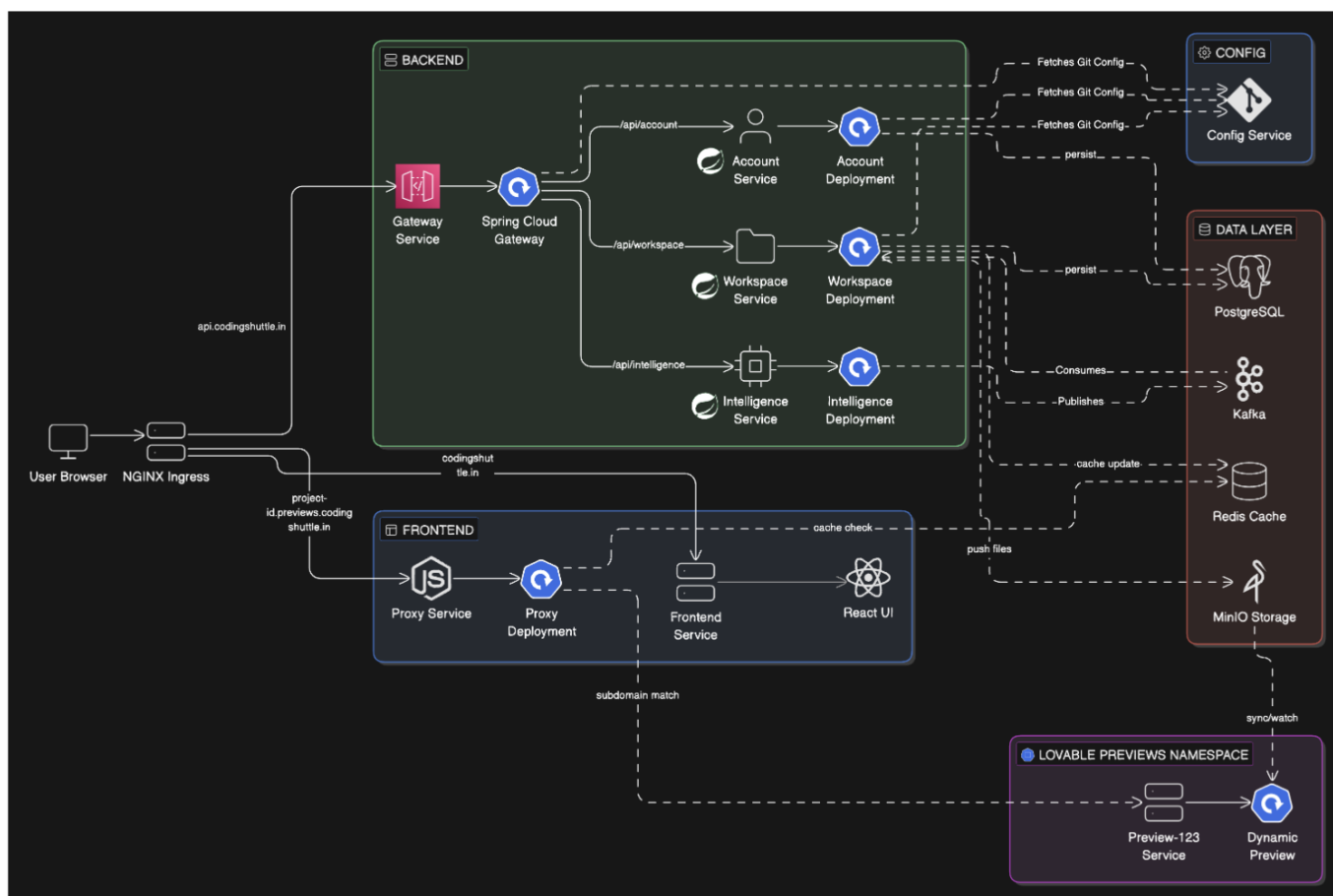
Incoming traffic passes through the NGINX Ingress controller, which distributes requests based on the host domain. API requests target `api.codingshuttle.in`, which proxies to the Spring Cloud Gateway. The Gateway uses Eureka service discovery to locate healthy instances of the Account, Workspace, and Intelligence services.

The diagram on the next page maps these relationships, detailing how data flows from the Gateway to the microservices, and how updates are persisted to the PostgreSQL database, MinIO object store, Redis cache, and Kafka broker.

Topic 6: LLD Blueprint - Microservices Diagram

Codingshuttle Ingress, Ports, & Gateway Network Blueprint

Overall Platform LLD Blueprint:



Topic 6: LLD Blueprint Blueprint Analysis

Decoupling microservices ports & ingress route details

The Low-Level Design blueprint maps the overall platform execution layout:

1. Ports Configuration: The services run on dedicated ports: Gateway on `8080`, Account Service on `9050` (mapping to context path `/account`), Workspace Service on `9020` (mapping to context path `/workspace`), Intelligence Service on `9030` (mapping to context path `/intelligence`), Config Service on `8888`, and Eureka on `8761`.
2. Config Syncing: On startup, each microservice calls the Config Service on port 8888. The Config Service queries the GitHub configuration repository and returns the application properties (datasource URLs, Kafka brokers, security secrets).
3. Database Connectivity: Account, Workspace, and Intelligence services connect to PostgreSQL. The Workspace Service also connects to MinIO on port 9000 to save project files, and the Proxy Service queries Redis on port 6379 for routing entries.
4. Wildcard Resolution: The Ingress Controller routes `\*.previews.codingshuttle.in` to the Proxy Service. The Proxy Service checks the Redis cache, retrieves the IP of the claimed runner container in the previews namespace, and forwards HTTP traffic directly.

This clear port separation allows multiple microservices to run concurrently on a single machine during local development without port collisions, while mirroring the cluster configuration.

Operational Ports: Keeping port allocations clean and routing traffic through context paths allows local developers to run the entire suite under Docker Compose using matching configurations.

Topic 6: LLD Blueprint - Sandbox & Sync Blueprint

Dynamic Preview sandbox multi-container isolation maps

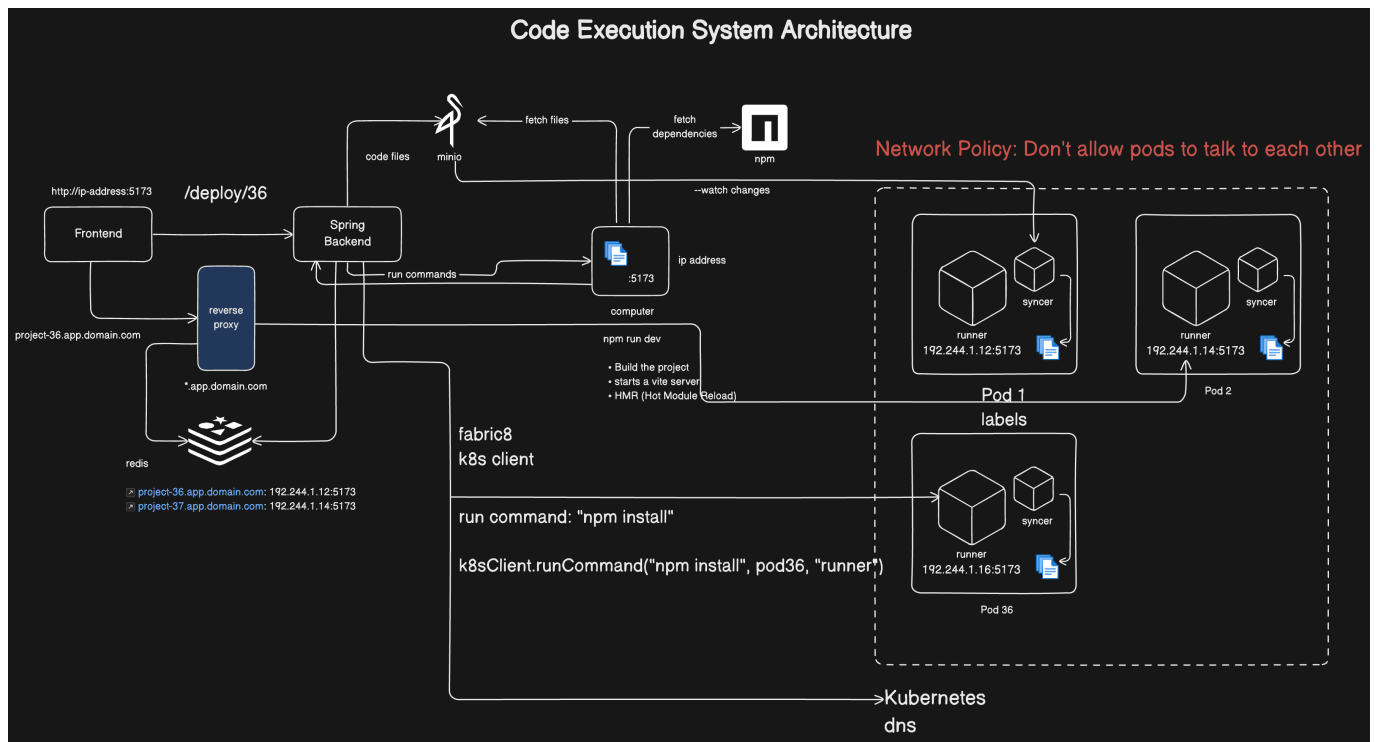
Exposing user-generated applications requires strict security boundaries and automated deployment pipelines. The LLD isolates user web runtimes inside a dedicated Kubernetes namespace (`lovable-previews`). This separates dynamic runtimes from core platform services, ensuring that custom code cannot crash the core cluster.

Each workspace is assigned a pre-warmed pod running a dual-container configuration. The `syncer` sidecar container handles file synchronization with the MinIO object store, while the `runner` container hosts the development server (e.g., Vite/Node.js). The diagram on the next page maps these relationships, detailing the volume mounts, MinIO sync loops, and the Proxy Service routing paths.

Topic 6: LLD Blueprint - Sandbox Diagram

Dual Container Sidecar & Volume Mounting blueprint

Sandbox Isolation LLD Blueprint:



Topic 6: LLD Blueprint Sandbox Blueprint Analysis

EmptyDir sharing, sidecar syncer & NPM compilation details

The Sandbox isolation Low-Level Design (LLD) maps the pod execution environment:

- Shared Volume Mount:** The pod configures an `emptyDir` volume, mounted at `/app` in both the `syncer` and `runner` containers. This shared directory serves as the local workspace filesystem.
- Syncer Sidecar Container:** Runs the MinIO client (`mc`). When a pod is claimed, the workspace service executes a `sync` command in this container to download the project files from the `projects/{id}` path in MinIO to `/app`. It then runs `mc mirror --watch` to listen for new file updates and sync them dynamically.
- Runner Container:** Runs the Node/Vite development server. Once files are synced, the workspace service triggers a shell execution command inside this container: `npm install && npm run dev -- --host 0.0.0.0 --port 5173`. The server hosts the code locally in the pod on port 5173.
- Proxy Routing:** When a request targets the preview subdomain, the Proxy Service queries Redis, retrieves the pod's IP, and proxies the HTTP traffic directly to the runner container on port 5173.

This dual-container configuration ensures that the code compiler container doesn't need external internet access to MinIO, reducing the security footprint of the sandbox environment.

Security Isolation: Since the pod has a dedicated network policy, it cannot access the main cluster control plane or other runner pods, securing the system against malicious user code execution.

Topic 6: LLD Code: Gateway JWT Security Filter

Reactive JWT Token validation & Gateway public route checks

The API Gateway validates authentication tokens locally using a global filter. Because Spring Cloud Gateway uses a reactive Netty runtime, filters must implement the non-blocking `GlobalFilter` interface. The filter intercepts incoming requests, checks them against public route configurations, extracts the Authorization header, validates the JWT signature, and passes the request if valid.

If the token is missing or invalid, the filter returns a `401 Unauthorized` response with a JSON payload, blocking unauthorized

traffic at the gate. This reactive security model prevents database calls and session lookups, ensuring high-performance routing.

This filter uses the `JwtGatewayService` to perform signature checks, verify expiration dates, and parse claims locally, keeping gateway operations fast and stateless.

Topic 6: LLD Code: GatewayJwtAuthFilter Source

Detailed filter method execution snapshot & Line-by-Line analysis

This class implements a reactive gateway filter. Here is a step-by-step technical breakdown of its execution flow:

- Line 1-3: Declarations and dependencies. `@Component` registers this class as a Spring Bean, and `@RequiredArgsConstructor` generates the constructor injecting private final dependencies.
- Line 8: `filter` method is the entrypoint. It receives the `ServerWebExchange` representing the reactive request context and `GatewayFilterChain` representing downstream filters.
- Line 9-11: Extracts the request URI path. It iterates through the list of public endpoints configured in `SecurityProperties` using `AntPathMatcher` to check if the route is public. If yes, it skips validation and calls `chain.filter(exchange)` to proceed immediately.
- Line 13-16: Extracts the `Authorization` header. If the header is missing or does not start with the 'Bearer ' prefix, it blocks execution and calls `sendErrorResponse` returning a 401 Unauthorized response to the client.
- Line 17-23: Extracts the token substring. It calls `jwtGatewayService.validateToken(token)` to verify the signature and expiration. If an exception is thrown, it captures the error and returns a 401 response with the exception detail. If successful, it calls `chain.filter(exchange)` to pass the request to the target microservice.

[GatewayJwtAuthFilter.java]

```
@Component @RequiredArgsConstructor
public class GatewayJwtAuthFilter implements GlobalFilter, Ordered {
    private final AntPathMatcher pathMatcher = new AntPathMatcher();
    private final SecurityProperties securityProperties;
    private final JwtGatewayService jwtGatewayService;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        String path = request.getURI().getPath();
        boolean isPublic = securityProperties.getPublicRoutes().stream()
            .anyMatch(pattern -> pathMatcher.match(pattern, path));
        if (isPublic) return chain.filter(exchange);

        String authHeader = request.getHeaders().getFirst("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            return sendErrorResponse(exchange, HttpStatus.UNAUTHORIZED, "Missing token");
        }
        String token = authHeader.substring(7);
        try {
            jwtGatewayService.validateToken(token);
        } catch (Exception e) {
            return sendErrorResponse(exchange, HttpStatus.UNAUTHORIZED, e.getMessage());
        }
        return chain.filter(exchange);
    }
}
```

Topic 6: LLD Code: Cross-Service Feign Interceptors

Feign Client Interceptor & security context propagation auto-configs

Microservice architectures typically lose user context during service-to-service calls. When the API Gateway proxies a request, it forwards the JWT token to the target microservice. However, if that microservice calls another service using a Feign client (e.g., the Workspace Service calling the Account Service to check subscription limits), the outbound Feign request lacks the authentication headers by default.

To resolve this, the Common Library registers a custom Feign `RequestInterceptor` bean. This interceptor extracts the active

JWT token from the `SecurityContextHolder` and injects it as a Bearer token into the outbound Feign header template. This maintains user context across internal calls without manual header propagation.

This class also registers the `jwtAuthFilter` bean, which is automatically configured in each microservice's filter chain to validate tokens locally.

Topic 6: LLD Code: SharedSecurityAutoConfiguration Source

Feign requestTemplate token interceptor beans mapping & Line-by-Line analysis

This configuration class propagates authentication contexts across internal services. Here is the step-by-step breakdown:

- Line 1: `@AutoConfiguration` designates this as a Spring Boot autoconfiguration class, automatically registered on service boot when imported.
- Line 3-11: Exposes the shared `AuthUtil` and `JwtAuthFilter` helper beans, injecting the generic `HandlerExceptionResolver` to capture filter errors and return JSON responses.
- Line 13-21: The `RequestInterceptor` bean definition. This interceptor executes before every Feign HTTP request.
- Line 15-18: Queries Spring Security's `SecurityContextHolder.getContext().getAuthentication()`. If authenticated, it extracts the credentials (the JWT token string).
- Line 19: If present, it injects the token into the outbound request template header under the key `Authorization` with prefix `Bearer `. This propagates the caller's context to the target service.

[SharedSecurityAutoConfiguration.java]

```
@AutoConfiguration
public class SharedSecurityAutoConfiguration {
    @Bean
    public AuthUtil authUtil() {
        return new AuthUtil();
    }

    @Bean
    public JwtAuthFilter jwtAuthFilter(AuthUtil authUtil,
        @Qualifier("handlerExceptionResolver") HandlerExceptionResolver hr) {
        return new JwtAuthFilter(authUtil, hr);
    }

    @Bean
    public RequestInterceptor requestInterceptor(){
        return requestTemplate -> {
            Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
            if (authentication != null && authentication.getCredentials() instanceof String token) {
                requestTemplate.header("Authorization", "Bearer " + token);
            }
        };
    }
}
```

Topic 6: LLD Code: Context Advisors & Context Injections

File tree gathering advisors mapping context prompts

To generate contextual code updates, the LLM must be aware of the project's current file structure. Passing the entire file tree in every prompt is inefficient, but the model needs path listings to identify which files to edit or create.

The Intelligence Service solves this using Spring AI's stream advisor framework. The `FileTreeContextAdvisor` intercepts prompt requests, fetches the active project's file list from the Workspace Service via Feign, formats it as a text tree structure, and injects it as a System Instruction before sending the query to OpenAI. This ensures the model generates accurate file paths matching the user's workspace.

The advisor dynamically intercepts request execution flows, modifying the target prompt structures before forwarding them to the LLM backend client, maintaining context mapping accuracy.

Topic 6: LLD Code: FileTreeContextAdvisor Source

StreamAdvisor client request augmentations & Line-by-Line analysis

This advisor class implements the `StreamAdvisor` interface to inject context into the prompt stream:

- Line 1: `@Component` registers the class as a Spring Bean, and `@RequiredArgsConstructor` generates the constructor injecting `WorkspaceClient`.
- Line 4-9: `adviseStream` method intercepts the active request stream. It extracts the `projectId` context parameter and calls `augmentRequestWithFileTree`.
- Line 12-14: Extracts instructions from the prompt, calls `workspaceClient.getFileTree(projId)` via Feign to retrieve the current file tree structure, and formats the output into a string block (`fileTree.toString()`).
- Line 16-18: Constructs a new system instruction system message passing the file tree context, appends the existing user conversation messages, and updates the prompt inside the request using a fluent mutation builder before calling the next advisor.

[FileTreeContextAdvisor.java]

```
@Component @RequiredArgsConstructor
public class FileTreeContextAdvisor implements StreamAdvisor {
    private final WorkspaceClient workspaceClient;

    @Override
    public Flux<ChatClientResponse> adviseStream(ChatClientRequest request,
                                                StreamAdvisorChain chain) {
        Map<String, Object> context = request.context();
        Long projectId = Long.parseLong(context.getOrDefault("projectId", 0).toString());
        ChatClientRequest augmentedRequest = augmentRequestWithFileTree(request, projectId);
        return chain.nextStream(augmentedRequest);
    }

    private ChatClientRequest augmentRequestWithFileTree(ChatClientRequest request, Long projId) {
        List<Message> incomingMessages = request.prompt().getInstructions();
        List<FileNode> fileTree = workspaceClient.getFileTree(projId).files();
        String fileTreeContext = "\n\n ---- FILE_TREE ----\n" + fileTree.toString();

        List<Message> allMessages = new ArrayList<>();
        allMessages.add(new SystemMessage(fileTreeContext));
        allMessages.addAll(incomingMessages);
        return request.mutate().prompt(new Prompt(allMessages, request.prompt().getOptions())).build();
    }
}
```

Topic 6: LLD Code: Prompt Streaming reactive endpoint

Reactive stream mapping triggers prompting loops

The prompt streaming engine uses Spring AI's `ChatClient` to execute streaming queries. It binds context advisors and tool calls, streaming results back while saving dialogue history:

Using Project ID method security permissions checks (`@PreAuthorize`), it ensures that the user is a valid member of the workspace before starting the chat session. It uses Spring AI's Fluent API model to configure the prompt instructions, parameter properties, custom advisors, and streaming hooks. The response is returned as a reactive `Flux<StreamResponse>`, allowing tokens to stream to the browser in real-time.

Using a reactive streaming pipeline prevents thread blocking during slow API calls to external AI services, allowing the system to support higher concurrent user volumes.

Topic 6: LLD Code: AIGenerationServiceImpl streamResponse Source

Streaming prompt response loop snapshot & Line-by-Line analysis

This service class coordinates reactive prompting. Here is the step-by-step logic breakdown:

- Line 1: `@PreAuthorize` verifies user authorization. The check calls the `@security.canEditProject` bean method.

- Line 3-6: Retrieves the `userId` from security context, retrieves or inserts a `ChatSession` record, and registers the parameter mapping maps passing the project and user identifiers.
- Line 7-10: Initializes a local buffer `fullResponseBuffer` to capture the streamed output, and binds the project tool interface (`CodeGenerationTools`).
- Line 11-19: Calls the chat client using the fluent builder pattern. It sets the system prompt instructions, the user prompt text, binds tools, and registers the custom `fileTreeContextAdvisor` advisor passing context parameters.
- Line 20-25: Calls `.stream().chatResponse()` to return a reactive stream. `doOnNext` appends each received chunk to the `fullResponseBuffer` buffer.
- Line 26-30: `doOnComplete` schedules the asynchronous execution of `finalizeChats` on a bounded elastic thread pool once the stream completes.
- Line 30-33: Converts the reactive chat response stream into a `Flux<StreamResponse>` to stream tokens directly to the client.

[AIGenerationServiceImpl.java (streamResponse method)]

```
@Override @PreAuthorize("@security.canEditProject(#projectId)")
public Flux<StreamResponse> streamResponse(String userMessage, Long projectId) {
    Long userId = authUtil.getCurrentUserId();
    ChatSession chatSession = createChatSessionIfNotExists(projectId, userId);
    Map<String, Object> advisorParams = Map.of("userId", userId, "projectId", projectId);
    StringBuilder fullResponseBuffer = new StringBuilder();
    CodeGenerationTools codeGenerationTools = new CodeGenerationTools(projectId, workspaceClient);

    return chatClient.prompt()
        .system(PromptUtils.CODE_GENERATION_SYSTEM_PROMPT)
        .user(userMessage)
        .tools(codeGenerationTools)
        .advisors(advisorSpec -> {
            advisorSpec.params(advisorParams);
            advisorSpec.advisors(fileTreeContextAdvisor);
        }).stream()
        .chatResponse()
        .doOnNext(response -> {
            if (response.getResults() != null && !response.getResults().isEmpty()) {
                String content = response.getResult().getOutput().getText();
                fullResponseBuffer.append(content);
            }
        })
        .doOnComplete(() -> {
            Schedulers.boundedElastic().schedule(() -> {
                finalizeChats(userMessage, chatSession, fullResponseBuffer.toString(), userId);
            });
        })
        .map(response -> {
            String text = response.getResult().getOutput().getText();
            return new StreamResponse(text != null ? text : "");
        });
}
```

Topic 6: LLD Code: AI Response parsing & event emission

Regex tagging parsing emitting Kafka updates

When the AI stream completes, the final text block is analyzed to extract generated code updates. The `LlmResponseParser` uses regular expressions to find `` tags containing path and content details.

For each file change, a `FileStoreRequestEvent` is published to Kafka. This event uses a unique `sagald` to ensure transactional integrity. The updates are processed asynchronously, allowing the chat stream to remain responsive without waiting for file writes or database updates to complete.

The parser handles both new file creation and partial modifications, translating LLM code suggestions into structured filesystem commands.

Topic 6: LLD Code: AIGenerationServiceImpl finalizeChats Source

Parsing file updates emitting Saga events & Line-by-Line analysis

This method parses the completed AI response and publishes file update events to the Kafka broker. Line-by-line analysis:

- Line 2-5: Persists the user message prompt and LLM output response to the `chat\_messages` table as `USER` and `ASSISTANT` records.
- Line 7: Calls `llmResponseParser.parseChatEvents` to parse the response text for file actions (e.g. `<file-edit>` tags).
- Line 8-15: Filters the parsed events. For each `FILE\_EDIT` action: generates a unique `sagald` (UUID string), creates a `FileStoreRequestEvent` containing the project ID, path, content, and user ID, and publishes it to the `file-store-request-event` Kafka topic.
- Line 16: Persists the parsed chat events list to database audits tables, linking them to the assistant message.

[AIGenerationServiceImpl.java (finalizeChats method)]

```
private void finalizeChats(String msg, ChatSession session, String text, Long userId) {
    chatMessageRepository.save(ChatMessage.builder()
        .chatSession(session).role(MessageRole.USER).content(msg).build());

    ChatMessage assistantMsg = chatMessageRepository.save(ChatMessage.builder()
        .chatSession(session).role(MessageRole.ASSISTANT).content(text).build());

    List<ChatEvent> events = llmResponseParser.parseChatEvents(text, assistantMsg);
    events.stream().filter(e -> e.getType() == ChatEventType.FILE_EDIT).forEach(e -> {
        String sagaId = UUID.randomUUID().toString();
        e.setSagaId(sagaId);
        FileStoreRequestEvent event = new FileStoreRequestEvent(
            session.getId().getProjectId(), sagaId, e.getFilePath(), e.getContent(), userId);
        kafkaTemplate.send("file-store-request-event", "project-" + session.getId().getProjectId(), event);
    });
    chatEventRepository.saveAll(events);
}
```

Topic 6: LLD Code: Kafka Saga File Consumer

Eventual workspace consistency consumer checking idempotence keys

The Workspace Service consumes `FileStoreRequestEvent`s from the Kafka broker. To ensure event processing is idempotent and prevent duplicate writes, the service checks the `processed\_events` table for the event's `sagald` before taking action.

If the event has not been processed, the service saves the file to MinIO, updates the file metadata in PostgreSQL, records the `sagald` in the `processed\_events` table, and publishes a `FileStoreResponseEvent` to the `file-store-responses` topic to complete the Saga transaction.

This mechanism guarantees eventual data consistency across the cluster, ensuring that temporary network cuts or duplicate messages do not cause corruption in the user's workspace.

Topic 6: LLD Code: FileStorageConsumer consumerFileEvent Source

Kafka listener processing workspace file events & Line-by-Line analysis

This consumer maintains data consistency using Kafka. Here is the step-by-step logic breakdown:

- Line 1: `@Transactional` joins the active database transaction context. If processing throws an exception, changes are rolled back.
- Line 2: `@KafkaListener` configures a Kafka consumer that listens to the `file-store-request-event` topic under consumer group `workspace-group`.
- Line 5-9: Idempotency Check. Queries the `processed\_events` table. If the `sagald` exists, the event has already been processed. The consumer skips execution and calls `sendResponse` to acknowledge success downstream.
- Line 11-16: Saves the file. It logs the event, calls `projectFileService.saveFile(projectId, path, content)` to save the file

metadata to PostgreSQL and the content to MinIO, registers the transaction ID in the `processed\_events` table, and sends a success acknowledgment event.

- Line 17-20: Exception Handler. If an exception occurs, it logs the error and sends a failure response containing the error message to complete the Saga rollback.

[FileStorageConsumer.java (consumerFileEvent method)]

```
@Transactional
@KafkaListener(topics = "file-store-request-event", groupId = "workspace-group")
public void consumerFileEvent(FileStoreRequestEvent requestEvent){
    // Idempotency check: If the event has already been processed, skip it
    if(processedEventRepository.existsById(requestEvent.sagaId())){
        log.info("Event with id {} already processed. Skipping.", requestEvent.sagaId());
        sendResponse(requestEvent, true, null);
        return;
    }

    try{
        log.info("Received event for project: {}, file: {}", requestEvent.projectId(), requestEvent.filePath());
        projectFileService.saveFile(requestEvent.projectId(), requestEvent.filePath(), requestEvent.content());
        processedEventRepository.save(new ProcessedEvent(requestEvent.sagaId(), LocalDateTime.now()));
        sendResponse(requestEvent, true, null);
    } catch(Exception e){
        log.error("Error saving file: {}", e.getMessage());
        sendResponse(requestEvent, false, e.getMessage());
    }
}
```

Topic 6: LLD Code: Kubernetes dynamic runner pod allocations

Claiming idle pods and configuring syncer commands

The Workspace Service manages runner allocations. It queries the cluster namespace for an `idle` pod, updates labels to `busy`, assigns the project ID, and runs setup and synchronization commands to initialize the sandbox environment:

1. Check Cache: It checks for an existing active pod assigned to the project. If found, it registers the route and returns.
2. Claim Pod: If no pod exists, it claims an idle pod by updating its labels to `status=busy` and `project-id={id}`.
3. Mirror Storage: It executes shell commands in the pod's `syncer` container to mirror files from MinIO to the shared volume.
4. Start Server: It runs `npm install` and starts the development server in the `runner` container, returning the preview URL.

By executing synchronization directly inside the pod's syncer sidecar, the core workspace microservice offloads compilation workloads from the main backend services.

Topic 6: LLD Code: KubernetesDeploymentServiceImpl deploy Source

Dynamic runner pod claiming entry loop & Line-by-Line analysis

This class claims Kubernetes sandbox pods dynamically. Line-by-line analysis:

- Line 1-4: Constructs the preview routing URL string using the `projectId` prefix and target root domain mapping.
- Line 6-10: Checks for an active pod. It calls `findActivePod(projectId)`. If found (non-null), the pod is already claimed. The method calls `registerRoute` to update the Redis mapping cache and returns a success response containing the preview URL.
- Line 11: If no pod is active, it calls `claimAndStartNewPod(projectId, domain, formattedUrl)` to claim an idle pod from the pre-warmed pool.
- Line 14-20: `findActivePod` details. Queries the Kubernetes cluster namespace using the Fabric8 client, filtering pods by labels (`project-id={id}` and `status=busy`). It verifies the pod phase is `Running` and returns the matching pod.

[KubernetesDeploymentServiceImpl.java (deploy method)]

```

public DeployResponse deploy(Long projectId) {
    String previewKey = "project-" + projectId;
    String domain = previewKey + "." + baseDomain;
    String formattedUrl = "https://" + domain + "/";

    Pod existingPod = findActivePod(projectId);
    if (existingPod != null) {
        registerRoute(domain, existingPod);
        return new DeployResponse(formattedUrl);
    }
    return claimAndStartNewPod(projectId, domain, formattedUrl);
}

private Pod findActivePod(Long projectId) {
    return client.pods().inNamespace(namespace)
        .withLabel("project-id", projectId.toString())
        .withLabel("status", "busy")
        .list().getItems().stream()
        .filter(pod -> pod.getStatus().getPhase().equals("Running"))
        .findFirst().orElse(null);
}

```

Topic 6: LLD Code: claimAndStartNewPod & execCommand Source**Claiming pod labels and executing syncer commands & Line-by-Line analysis**

This method claims an idle pod and initializes the code compiler. Line-by-line analysis:

- Line 2-4: Queries the namespace for a pod labeled `status=idle`. If none exist, it throws a runtime exception.
- Line 6-10: Claims the pod. It updates the pod labels (`status=busy` and `project-id={id}`) to bind it to the project.
- Line 13-18: Downloads workspace files. It runs a shell command inside the pod's `syncer` container to download the project files from MinIO, then launches `mc mirror --watch` in the background to sync future modifications.
- Line 19-21: Starts the development server. It runs `npm install` and starts the Vite server in the `runner` container on port 5173.
- Line 22-24: Updates the Redis routing cache with the pod's IP and returns the preview URL.
- Line 25-28: Error Handler. If an execution fails, it deletes the pod to prevent resource leaks and throws a runtime exception.

[KubernetesDeploymentServiceImpl.java (claimAndStartNewPod & exec)]

```

private DeployResponse claimAndStartNewPod(Long projectId, String domain, String formattedUrl) {
    Pod pod = client.pods().inNamespace(namespace).withLabel("status", "idle")
        .list().getItems().stream().findFirst()
        .orElseThrow(() -> new RuntimeException("No idle runners available."));

    String podName = pod.getMetadata().getName();
    client.pods().inNamespace(namespace).withName(podName).edit(p -> {
        p.getMetadata().getLabels().put("status", "busy");
        p.getMetadata().getLabels().put("project-id", projectId.toString());
        return p;
    });

    try {
        String syncCmd = String.format("rm -rf /app/* && mc mirror --overwrite myminio/projects/%d/ /app/", projectId);
        execCommand(podName, "syncer", "sh", "-c", syncCmd);

        String watchCmd = String.format("nohup mc mirror --overwrite --watch myminio/projects/%d/ /app/ > /app/sync.log 2>&1 &",
projectId);
        execCommand(podName, "syncer", "sh", "-c", watchCmd);

        String startCmd = "npm install && nohup npm run dev -- --host 0.0.0.0 --port 5173 > /app/dev.log 2>&1 &";
        execCommand(podName, "runner", "sh", "-c", startCmd);

        Pod updatedPod = client.pods().inNamespace(namespace).withName(podName).get();
        registerRoute(domain, updatedPod);
        return new DeployResponse(formattedUrl);
    } catch (Exception e) {
        client.pods().inNamespace(namespace).withName(podName).delete();
        throw new RuntimeException("Deployment failed: " + e.getMessage(), e);
    }
}

```